MedRoute - Entity Relationship Diagram (ERD)

PostgreSQL 16 + PostGIS + pgRouting | Pune edition

node_id: nearest road-graph vertex (N : 1)

users	
PK **user_id**	BIGINT
name	VARCHAR
email	VARCHAR (unique)
password_hash	VARCHAR (BCrypt)
role	ADMIN | AMBULANCE | HOSPITAL

1 — 0..1 *drives*

ambulances	
PK **ambulance_id**	BIGINT
vehicle_number	VARCHAR
FK driver_id	-> users
latitude	DOUBLE
longitude	DOUBLE
status	VARCHAR
FK node_id	-> ways_vertices_pgr
geom	GEOMETRY(Point,4326)

1 — N *handles*

emergency_requests	
PK **request_id**	BIGINT
FK ambulance_id	-> ambulances
FK patient_id	-> patients
FK hospital_id	-> hospitals
severity	VARCHAR
status	lifecycle state
created_at	TIMESTAMP
closed_at	TIMESTAMP
eta_seconds	INT

N — 1 *sent to*

hospitals	
PK **hospital_id**	BIGINT
name	VARCHAR
latitude	DOUBLE
longitude	DOUBLE
status	ACTIVE | INACTIVE
FK node_id	-> ways_vertices_pgr
geom	GEOMETRY(Point,4326)

1 — 1 *has stock*

hospital_resources	
PK,FK **hospital_id**	-> hospitals
icu_available	INT
emergency_beds	INT
ventilators	INT
blood_units	INT
updated_at	TIMESTAMP

1 — N *per blood group*

hospital_blood_stock (optional)	
PK,FK **hospital_id**	-> hospitals
PK **blood_group**	VARCHAR(3)
units	INT

ambulances 1 — N *carries* — N patients 1

N — 1 *is subject of* — emergency_requests

emergency_requests 1 — N *has route* — routes

patients	
PK **patient_id**	BIGINT
FK ambulance_id	-> ambulances
age	INT
blood_group	VARCHAR(3)
trauma_level	VARCHAR
heart_rate	INT
spo2	INT

routes	
PK **route_id**	BIGINT
FK request_id *	-> emergency_requests
source	BIGINT (vertex)
destination	BIGINT (vertex)
distance	DOUBLE (m)
traffic_weight	DOUBLE
estimated_time	DOUBLE (s)
path_json	JSONB

patients 1 — N *records* — N telemetry_log

telemetry_log	
PK **id**	BIGINT
FK patient_id	-> patients
heart_rate	INT
spo2	INT
recorded_at	TIMESTAMP

ways_vertices_pgr (pgRouting)	
PK **id**	BIGINT
lon	DOUBLE
lat	DOUBLE
the_geom	GEOMETRY

N — N *edge reference*

traffic_events	
PK **id**	BIGINT
FK edge_id	-> ways.gid
multiplier	DOUBLE (1.0/1.6/2.5)
started_at	TIMESTAMP
ended_at	TIMESTAMP

N — 1

ways (pgRouting)	
PK **gid**	BIGINT
source / target	vertex ids
length_m	DOUBLE
cost / reverse_cost	DOUBLE

Legend and notes

PK FK primary key / foreign key. 1, N, 0..1 = cardinality. Solid line = relationship between MedRoute tables.

Dashed line / dashed box = link to tables created by osm2pgrouting (road graph).

* routes.request_id is added here to link a stored route to its emergency; the plan lists routes without an explicit foreign key.
Fields come from the plan's database design (Section 6), including its NEW fields: node_id, geom, updated_at, closed_at, eta_seconds, path_json, telemetry_log, traffic_events and the optional hospital_blood_stock. Data types are suggested. All vitals, traffic and hospital resource values are simulated.

MedRoute

Software Requirements Specification (SRS)

Emergency Smart Ambulance Routing and Hospital Resource Dispatcher - Pune Edition

Item	Detail
Document version	1.0 (derived from the MedRoute Complete Build Plan)
Project type	MCA project
Technology	Java 21, Spring Boot 3.x, PostgreSQL 16 + PostGIS + pgRouting, React (Vite), Leaflet, STOMP/WebSocket
Map area	Pune, Maharashtra (center about 18.5204, 73.8567)
Scope note	Operational decision-support and coordination platform. All patient vitals, traffic and hospital resource numbers are simulated. It does not diagnose patients.

1. Introduction

1.1 Purpose

This document specifies the functional and non-functional requirements of MedRoute. It is the agreed basis for design, implementation, testing and the final report. The companion ERD and use case diagram appear in the same file.

1.2 Scope

MedRoute connects an ambulance, its patient telemetry, live road conditions and hospital resources in one system, so that the ambulance is sent to a hospital that can actually treat the patient quickly, not simply the nearest one. The system provides secure role-based access, routing on the real Pune road network (A* and Dijkstra with dynamic traffic weights), rule-based hospital selection, an emergency lifecycle, simulators for traffic, GPS and vitals, real-time dashboards and admin reports. Out of scope: diagnosis, real sensors, live traffic APIs, mobile apps and cloud deployment (listed as future enhancements).

1.3 Definitions and abbreviations

Term	Meaning
A*	Shortest-path algorithm using f(n) = g(n) + h(n); h is haversine distance divided by maximum speed.
Dijkstra	Shortest-path algorithm equal to A* with heuristic 0; used as reference.
OSM	OpenStreetMap, source of Pune road data and hospital coordinates.
pgRouting / PostGIS	PostgreSQL extensions for routing and spatial data.
Traffic multiplier	Factor applied to edge travel time: normal 1.0, moderate 1.6, heavy 2.5, blocked (edge ignored).
ETA	Estimated time of arrival in seconds, taken from the route cost.
JWT	JSON Web Token used for stateless authentication.
STOMP / SockJS	Messaging protocol over WebSocket and its fallback transport.
SpO2	Blood oxygen saturation (simulated value).

1.4 References

- MedRoute Complete Build Plan - Pune Edition (source document).
- Companion ERD and use case diagram in this PDF.

- OpenStreetMap, pgRouting and osm2pgrouting documentation.

2. Overall Description

2.1 Product perspective

MedRoute is a standalone three-tier web application: a React single-page client, a Spring Boot REST and WebSocket server, and a PostgreSQL database with PostGIS and pgRouting. The road graph is imported once from OSM and then loaded into memory as an adjacency list, so MedRoute's own routing code runs independently of pgRouting (which is used only for verification).

2.2 User classes

Role	Main screens	Key actions
ADMIN	Admin dashboard, reports	Manage users, hospitals and ambulances; view emergency history; tune scoring weights; compare A* and Dijkstra; block or slow roads.
AMBULANCE	Ambulance dashboard	Start an emergency, enter patient data, see recommended hospital, route, ETA and live status, close the case.
HOSPITAL	Hospital ER dashboard	Update resources, see incoming patients with live vitals and ETA, acknowledge alerts.
Simulators	None (internal)	Secondary actors: traffic, GPS and vitals simulators that feed live data.

2.3 Operating environment

- Server: JDK 21 (LTS), Maven 3.9+, Spring Boot 3.x on port 8080.
- Database: PostgreSQL 16 with PostGIS and pgRouting, run through Docker Compose.
- Client: modern desktop browser; Node.js 20 LTS+ for the Vite build; Leaflet with OpenStreetMap tiles.

2.4 Constraints

- Road data limited to the Pune bounding box (west 73.75, south 18.40, east 73.98, north 18.65); may be trimmed if the file is too large.
- Real hospital names and locations are used; ICU, ventilator and blood counts are invented and must be labelled as simulated.
- Delivery follows a 10-week plan; routing and hospital selection are completed before UI polish.

2.5 Assumptions and dependencies

- Each hospital and ambulance station can be snapped to a nearest graph vertex (node_id).
- Maximum road speed in the graph is known so the A* heuristic never overestimates.
- OSM tile servers are reachable from the browser during demos.

3. Specific Requirements

3.1 External interface requirements

User interface. React pages: Login, AmbulanceDashboard, HospitalDashboard, AdminDashboard; components MapView, PatientCard, ResourcePanel and AlertList. The map shows hospital markers, a moving ambulance marker and a route polyline coloured by traffic.

Software interface. REST API under /api (auth, ambulances, hospitals, emergencies, routes, simulation, reports); PostgreSQL via JPA/JDBC; pgRouting queries for verification.

Communication interface. STOMP over WebSocket at /ws with SockJS fallback, JWT sent on connect. Topics: /topic/ambulance/{id}/status, /topic/ambulance/{id}/route, /topic/hospital/{id}/incoming, /topic/patient/{id}/vitals, /topic/alerts. Envelope: { type, emergencyId, timestamp, data }.

3.2 Functional requirements

Priority: M = Must, S = Should.

Authentication and access control

ID	Requirement	Priority
FR-AUTH-01	The system shall authenticate users by email and password through POST /api/auth/login and return a JWT and the user's role.	M
FR-AUTH-02	Passwords shall be stored only as BCrypt hashes.	M
FR-AUTH-03	The system shall enforce role-based access for ADMIN, AMBULANCE and HOSPITAL: wrong role returns 403; missing, expired or tampered JWT returns 401.	M
ID	**Requirement**	**Priority**
FR-AUTH-04	On startup the system shall seed one admin, three ambulance users and five hospital users.	S

Master data management

ID	Requirement	Priority
FR-DATA-01	Admin shall create, view and update users, hospitals and ambulances.	M
FR-DATA-02	Hospital staff and admin shall update ICU beds, emergency beds, ventilators and blood units; the system shall record updated_at. Optional per-blood-group stock shall be supported.	M
FR-DATA-03	Ambulance staff and admin shall update ambulance status and position.	M
ID	**Requirement**	**Priority**
FR-DATA-04	Every hospital and ambulance station shall be snapped to its nearest road-graph vertex and store node_id.	M
FR-DATA-05	All inputs shall be validated through DTO annotations; errors shall use one consistent API error format via a global exception handler.	M

Road network and routing

ID	Requirement	Priority
FR-RTE-01	The system shall import Pune OSM road data into pgRouting tables (ways, ways_vertices_pgr).	M
FR-RTE-02	At startup the system shall load edges into an in-memory adjacency list with node id, latitude and longitude, and edge length, base speed and traffic multiplier (default 1.0).	M
FR-RTE-03	Edge cost in seconds shall be (length / speed) x trafficMultiplier; a blocked edge shall be skipped.	M
ID	**Requirement**	**Priority**
FR-RTE-04	The system shall implement A* with an admissible heuristic (haversine distance divided by maximum speed) and Dijkstra as reference, returning path, total cost, ETA and nodes visited.	M
FR-RTE-05	The system shall expose GET /api/routes/compare returning nodes visited and time taken for A* and Dijkstra; A* must return the same cost with fewer nodes visited.	M
FR-RTE-06	When traffic changes, the system shall recompute the route from the ambulance's current node (reroute).	M
FR-RTE-07	Own-algorithm path costs shall match pgr_dijkstra on verification pairs (10 random pairs).	S
FR-RTE-08	If no path exists the system shall return a clear no-path result.	M

Hospital selection

ID	Requirement	Priority
FR-SEL-01	Hard filters: hospital ACTIVE; at least 1 ICU bed if trauma is critical; at least 1 emergency bed; ventilator available if SpO2 is below the threshold (default 90); blood available for the patient's group. A hospital failing any filter is rejected.	M
FR-SEL-02	Remaining hospitals shall be scored (lower is better): wEta x etaNorm + wTraffic x trafficNorm + wResource x (1 - resourceMatch), each term normalized 0-1; default weights 0.5, 0.2, 0.3.	M
FR-SEL-03	The system shall return a ranked list with the reason each rejected hospital was rejected.	M
ID	**Requirement**	**Priority**
FR-SEL-04	If every hospital is filtered out, the system shall relax the least critical filter and raise a visible alert.	M
FR-SEL-05	Admin shall be able to change scoring weights.	S
FR-SEL-06	The system shall re-evaluate the choice when resources or traffic change materially, with a cooldown to prevent flipping.	S

Emergency workflow

ID	Requirement	Priority
FR-EMG-01	An ambulance user shall create an emergency (POST /api/emergencies); the system shall assign the nearest free ambulance.	M
FR-EMG-02	Lifecycle states: CREATED, AMBULANCE_ASSIGNED, PATIENT_PICKED, HOSPITAL_SELECTED, EN_ROUTE, ARRIVED, CLOSED. Invalid transitions shall be rejected.	M
FR-EMG-03	The ambulance user shall submit patient age, blood group, trauma level, heart rate and SpO2.	M
ID	**Requirement**	**Priority**
FR-EMG-04	Starting a trip shall compute the route, store route and ETA, and begin trip simulation.	M
FR-EMG-05	Closing a case (ambulance or hospital) shall record closed_at and free the ambulance.	M

Simulation

ID	Requirement	Priority
FR-SIM-01	TrafficSimulator shall change multipliers (1.0, 1.6, 2.5, blocked) on selected Pune corridors every 5-10 seconds.	M
FR-SIM-02	GpsSimulator shall move each active ambulance node by node along its route (one task per ambulance).	M
FR-SIM-03	VitalsSimulator shall generate heart rate and SpO2 with small random drift per patient and occasionally a warning value.	M
ID	**Requirement**	**Priority**
FR-SIM-04	Simulators shall have start/stop controls and a fixed random seed option for repeatable demos.	M
FR-SIM-05	Admin shall block or slow a road on demand (POST /api/simulation/traffic/block).	M
FR-SIM-06	The UI and reports shall label vitals, traffic and hospital resources as simulated.	M

Real-time updates and alerts

ID	Requirement	Priority
FR-RT-01	The system shall provide STOMP over WebSocket with SockJS fallback; the socket shall be authenticated with the JWT and subscriptions checked by role.	M
FR-RT-02	The system shall publish position, route and ETA, incoming-patient, vitals and alert messages to the topics listed in 3.1.	M
FR-RT-03	Messages shall use the envelope { type, emergencyId, timestamp, data }.	M
ID	**Requirement**	**Priority**
FR-RT-04	Publishing shall occur only on meaningful change (new position, out-of-range vital, ETA change of more than a few seconds).	M
FR-RT-05	Alerts shall be raised for low SpO2, blocked road, hospital resource change and reroute.	M
FR-RT-06	Changes shall appear on the other role's screen within about 1-2 seconds.	M

User interface and maps

ID	Requirement	Priority
FR-UI-01	The client shall provide a login page and role-based routing to the matching dashboard.	M
FR-UI-02	Ambulance dashboard: start emergency, enter patient data, recommended hospital, route, countdown ETA and live status.	M
FR-UI-03	Hospital dashboard: update resources, view incoming patient cards (vitals, blood group, trauma level, ETA) updating live, acknowledge alerts.	M
ID	**Requirement**	**Priority**
FR-UI-04	Admin dashboard: manage users, hospitals and ambulances, scoring weights and traffic controls.	M
FR-UI-05	Leaflet map centered on Pune with hospital markers, moving ambulance marker, route polyline and a traffic overlay coloured by multiplier (green, orange, red).	M
FR-UI-06	The UI shall show loading and error states.	S

Reports and history

ID	Requirement	Priority
FR-RPT-01	Admin shall view emergency history filtered by date (GET /api/reports/emergencies).	M
FR-RPT-02	Reports shall include average ETA and hospital usage.	S
FR-RPT-03	The system shall present an A* vs Dijkstra results table for 20 Pune origin-destination pairs.	M
ID	**Requirement**	**Priority**
FR-RPT-04	Vitals history shall be kept in telemetry_log and traffic changes in traffic_events.	S

3.3 Non-functional requirements

ID	Requirement	Priority
NFR-PER-01	Route computation on the Pune graph shall complete fast enough for interactive use (target: under 1 second on development hardware); the graph is held in memory.	M
NFR-PER-02	Live updates shall reach dashboards within about 1-2 seconds.	M
NFR-SEC-01	BCrypt password hashing, JWT authentication, role checks on every endpoint and every WebSocket subscription; secrets kept out of the repository.	M
NFR-REL-01	Shared state shall be thread-safe (ConcurrentHashMap, AtomicReference or volatile); no plain HashMap shared between threads; one writer per value.	M
NFR-REL-02	Simulators shall run 10+ minutes without exceptions; route computation during traffic updates shall cause no errors or lost updates.	M

ID	Requirement	Priority
NFR-REL-03	The client shall handle a dropped WebSocket connection gracefully (reconnect and resynchronize).	S
NFR-USE-01	Dashboards shall be readable at a glance during a live demo; messages are routed in the client by message type.	S
NFR-MNT-01	Code shall be modular by package (auth, ambulance, patient, hospital, routing, selection, emergency, simulation, websocket, common) with JUnit 5 and Mockito tests covering routing, selection, security, emergency flow, concurrency and WebSocket.	M
NFR-POR-01	The database shall run through Docker Compose and a new machine shall be able to run the project from the README alone.	M
NFR-SAF-01	The system supports coordination only; it shall not present output as medical diagnosis.	M

3.4 Business rules

- Time-based cost (not distance) is used so ETA is a direct output of the route.
- Selection is never nearest-first: hard filters first, then weighted score of ETA, traffic and resource match.
- Example scenario: Hospital A (closest, no ICU) is rejected; Hospital C (heavy traffic, multiplier 2.5) scores worse; Hospital B (slightly farther, ICU and blood available) is selected.

3.5 Data requirements

The logical data model is shown in the ERD: users, ambulances, patients, hospitals, hospital_resources, optional hospital_blood_stock, emergency_requests, routes, telemetry_log and traffic_events, linked to the pgRouting tables ways and ways_vertices_pgr through node_id and edge_id.

4. Acceptance and Verification

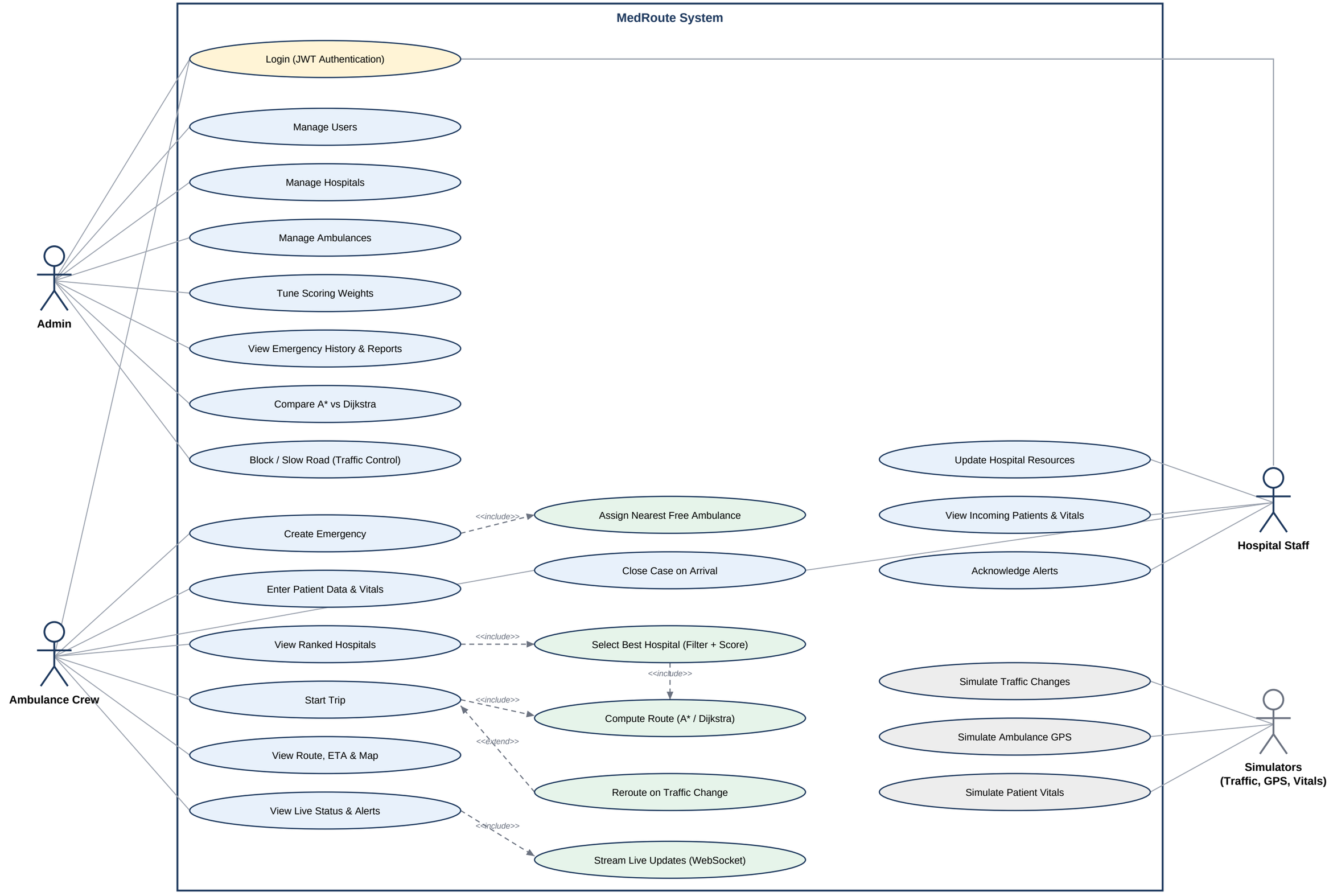
Area	Verification
Routing	Hand-computed small graph; A* cost equals Dijkstra; blocked edge forces detour; no-path case; A* visits fewer nodes on Pune pairs.
Selection	Example scenario picks Hospital B; no-ICU hospital rejected for a critical patient; all-rejected path; weight change flips the winner.
Security	Wrong password rejected; AMBULANCE cannot call admin APIs; expired or tampered JWT returns 401.
Emergency flow	States advance in order; closing frees the ambulance; invalid transition rejected.
Concurrency / WebSocket	No exceptions under concurrent traffic updates; hospital topic receives vitals; unauthorized subscription refused.
End to end	Create emergency, select hospital, trip, block road, reroute, arrive, close.

5. Limitations and Future Enhancements

- All vitals, traffic and hospital resources are simulated; no real sensors or live traffic feed.
- Future: live traffic API, predictive ETA, mobile app, cloud deployment (only after all 10 phases pass).

MedRoute - Use Case Diagram

Actors: Admin, Ambulance Crew, Hospital Staff, Simulators



Legend: *Authentication (required before every other use case)* · *User-initiated use case* · *System-performed use case* · *Simulator-driven use case*

Solid line = actor takes part in use case. Dashed arrow = <<include>> (always used) or <<extend>> (optional behavior). Simulators are secondary actors; all data is simulated.